Route Parameters and Query Strings in ASP.NET Core Web API

When developing RESTful services using ASP.NET Core Web API, it's common for the client to send data dynamically to the server, for example:

- ⇨ Fetching order details by order ID.
- ⇨ Retrieving a specific employee record.
- ⇨ Searching for products by category.

ASP.NET Core provides two main ways to receive such data:

- ➔ Route Data
- ➔ Query Strings

Both mechanisms help our API capture values directly from the request URL, but they serve different purposes.

Route Data: it is used when the value is an essential part of the resource path, for example, /api/employees/5 directly identifies the employee with ID 5. Use Route Data for mandatory and identity-based parameters (like IDs).

Query Strings: when the values are optional filters or modifiers, for example, /api/employees?department=HR&sortBy=name retrieves in the HR department sorted by name. Use Query strings for optional or filtering parameters (like search, filter or sort options).

PROJECT SETUP

<u>Step 1: Create Project</u>

```
dotnet new webapi -n LibraryApi1 -f net8.0

cd LibraryApi1
```

<u>Step 2: Install Packages (Version 8.0.0)</u>

1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

Step 3: Create Model (Book.cs)

Path: LibraryApi/Models/Book.cs

Book.cs

Models > Book.cs > ...

```
namespace LibraryApi1.Models
{
    0 references
    public class Book
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Title { get; set; } = string.Empty;
        0 references
        public string Author { get; set; } = string.Empty;
        0 references
        public string Genre { get; set; } = string.Empty;
        0 references
        public int PublishedYear { get; set; }
        0 references
        public bool IsAvailable { get; set; }
        0 references
        public decimal Price { get; set; }
    }
}
```

Step 4: Create In-Memory Data (BookData.cs)

Path: LibraryApi1/Data/BookData.cs

BookData.cs

Data > BookData.cs > ...

```
using LibraryApi1.Models;

namespace LibraryApi1.Data
{
    0 references
    public static class BookData
    {
        0 references
        public static List<Book> Books = new List<Book>
        {
            new Book
            {
                Id = 1,
                Title = "The Hobbit",
                Author = "J.R.R. Tolkien",
                Genre = "Fantasy",
                PublishedYear = 1937,
                IsAvailable = true,
                Price = 15.99m
            },
            new Book
            {
                Id = 2,
                Title = "1984",
                Author = "George Orwell",
                Genre = "Dystopian",
                PublishedYear = 1949,
                IsAvailable = false,
                Price = 12.99m
            },
```

```
            new Book
            {
                Id = 3,
                Title = "Pride and Prejudice",
                Author = "Jane Austen",
                Genre = "Romance",
                PublishedYear = 1813,
                IsAvailable = true,
                Price = 9.99m
            },
            new Book
            {
                Id = 4,
                Title = "The Great Gatsby",
                Author = "F. Scott Fitzgerald",
                Genre = "Classic",
                PublishedYear = 1925,
                IsAvailable = true,
                Price = 11.99m
            },
            new Book
            {
                Id = 5,
                Title = "Harry Potter",
                Author = "J.K. Rowling",
                Genre = "Fantasy",
                PublishedYear = 1997,
                IsAvailable = true,
                Price = 19.99m
            },
        };
    }
}
```

Step 5: Configure Program.cs (Minimal Setup)

Path: LibraryApi1/Program.cs

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

ROUTE PARAMETERS

- ➔ Single Route Parameter (Get by ID)
- ➔ Multiple Route Parameters

Single Route Parameter (Get by ID)

We are going to use [HttpGet("{id}")] to fetch book by ID.

Path: LibraryApi1/Controllers/BooksController.cs

```
using LibraryApi1.Data;
using LibraryApi1.Models;
using Microsoft.AspNetCore.Mvc;

namespace LibraryApi1.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    0 references
    public class BooksController : ControllerBase
    {
        // SINGLE ROUTE PARAMETER - Get book by ID
        // URL: GET /api/books/3
        [HttpGet("{id}")]
        0 references
        public ActionResult<Book> GetBookById(int id)
        {
            var book = BookData.Books.FirstOrDefault(b => b.Id == id);

            if(book == null)
            {
                return NotFound($"Book with ID {id} not found.");
            }
            return Ok(book);
        }

    }
}
```

How it works:

- *{id}* in route captures values from URL
- ASP.NET Core automatically binds to *id* parameter
- If book not found → *404* with message

Multiple Route Parameters

We are going to use *[HttpGet("genre/{genre}/author/{author}")]* to filters Books.

Path: LibraryApi1/Controllers/BooksController.cs (update)

```
// MULTIPLE ROUTE PARAMETERS - Get books by Genre and Author
// URL: GET /api/books/genre/Fantasy/author/Tolkien
[HttpGet("genre/{genre}/author/{author}")]
public ActionResult<IEnumerable<Book>>
0 references
GetBooksByGenreAndAuthor(string genre, string author)
{
    var filteredBooks = BookData.Books.Where(
        b => b.Genre
            .Equals(genre, StringComparison.OrdinalIgnoreCase)
        &&
        b.Author
        .Contains(author, StringComparison.OrdinalIgnoreCase)
    ).ToList();

    if (!filteredBooks.Any())
    {
        """"
        return NotFound
    ($"No books found with Genre '{genre}' and Author containing '{author}'.");
    }

    return Ok(filteredBooks);
}
```

How it works:

- Route template: "genre/{genre}/author/{author}"
- Both parameters captured from URL path
- Case-insensitive comparison

QUERY STRINGS

- ➔ Single Query String Parameter
- ➔ Multiple Query String Parameters

<u>Single Query String Parameter</u>

We will search data by a query through a string. We will use *[HttpGet("search")]* with *[FromQuery] string genre* to search a book by genre. Where url would be like > URL: */api/books/search?genre=Fantasy*

Path: LibraryApi1/Controllers/BooksController.cs (update)

```
// SINGLE QUERY STRING - Search by Genre
// URL: GET /api/books/search?genre=Fantasy
[HttpGet("search")]
0 references
public ActionResult<IEnumerable<Book>> SearchByGenre([FromQuery] string genre)
{
    var filteredBooks = BookData.Books.Where(b => b.Genre
        .Equals(genre, StringComparison.OrdinalIgnoreCase)).ToList();

    if (!filteredBooks.Any())
        """
    {
        return NotFound($"No books found in Genre '{genre}'.");
    }

    return Ok(filteredBooks);
}
```

How it works:

- *[FromQuery]* tells ASP.NET Core to look in query string
- URL format: */api/books/search?genre=Fantasy*
- Parameter is OPTIONAL (if missing, genre = null)

<u>Multiple Query String Parameters</u>

We are going to filter books using 3 query parameters: Gender, Department and City, where our predicted URL like:

GET /api/books/filter?genre=Fantasy&author=Rowling&isAvailable=true

Path: LibraryApi1/Controllers/BooksController.cs (update)

```
// MULTIPLE QUERY STRINGS - Filter by Genre, Author and Availablity
// URL: GET /api/books/filter?genre=Fantasy&author=Rowling&isAvailable=true
[HttpGet("filter")]
0 references
public ActionResult<IEnumerable<Book>> FilterBooks (
    [FromQuery] string? genre,
    [FromQuery] string? author,
    [FromQuery] bool? isAvailable
)
{
    var query = BookData.Books.AsQueryable();

    if (!string.IsNullOrEmpty(genre))
    {
        query = query.Where(b => b.Genre.Equals(genre,
            StringComparison.OrdinalIgnoreCase));
    }

    if (!string.IsNullOrEmpty(author))
    {
        query = query.Where(b => b.Author.Contains(author,
            StringComparison.OrdinalIgnoreCase));
    }

    if (isAvailable.HasValue)
    {
        query = query.Where(b => b.IsAvailable == isAvailable.Value);
    }

    var results = query.ToList();

    if (!results.Any())
    {
        return NotFound("No books match the provided search criteria.");
    }

    return Ok(results);
}
```

How it works:

- ? makes parameters optional (nullable types)
- Order of parameters in URL doesn't matter
- Multiple parameters separated by &

COMPLEX TYPE BINDING with [FromQuery]

STEPS:

- Create Filter Model
- Use Complex Type in Controller

Step 1: Create Filter Model

Path: LibraryApi1/Models/BookSearchFilter.cs

```
BookSearchFilter.cs ×
Models > BookSearchFilter.cs > ...
namespace LibraryApi1.Models
{
    0 references
    public class BookSearchFilter
    {
        0 references
        public string? Genre { get; set; }
        0 references
        public string? Author { get; set; }
        0 references
        public bool? IsAvailable { get; set; }
        0 references
        public decimal? MinPrice { get; set; }
        0 references
        public decimal? MaxPrice { get; set; }
    }
}
```

Step 2: Use Complex Type in Controller

Path: LibraryApi1/Controllers/BooksController.cs (update)

```
// COMPLEX TYPE BINDING - Using Model Class for many filters
// URL: GET /api/books/advanced-search?genre=Fantasy&author=Rowling&minPrice=10&maxPrice=20
[HttpGet("advanced-search")]
0 references
public ActionResult<IEnumerable<Book>> AdvancedSearch([FromQuery] BookSearchFilter filter)
{
    var query = BookData.Books.AsQueryable();

    if (!string.IsNullOrEmpty(filter.Genre))
    {
        query = query.Where(b => b.Genre.Equals(filter.Genre, StringComparison.OrdinalIgnoreCase));
    }

    if (!string.IsNullOrEmpty(filter.Author))
    {
        query = query.Where(b => b.Author.Contains(filter.Author, StringComparison.OrdinalIgnoreCase));
    }

    if (filter.IsAvailable.HasValue)
    {
        query = query.Where(b => b.IsAvailable == filter.IsAvailable.Value);
    }

    if (filter.MinPrice.HasValue)
    {
        query = query.Where(b => b.Price >= filter.MinPrice.Value);
    }

    if (filter.MaxPrice.HasValue)
    {
        query = query.Where(b => b.Price <= filter.MaxPrice.Value);
    }

    var results = query.ToList();

    if (!results.Any())
    {
        return NotFound("No books match the provided search criteria.");
    }

    return Ok(results);
}
```

By default, complex types bind from request body. Using *[FromQuery]* forces binding from query string.

COMBINING ROUTE PARAMETERS AND QUERY STRINGS

Here in this part we will use, Route parameter (Genre) + Query strings (MinPrice, MaxPrice, IsAvailble) where the URL like:

GET /api/books/genre/Fantasy?minPrice=10&maxPrice=20&isAvailable=true

Path: LibraryApi1/Controllers/BooksController.cs (update)

```
// COMBINED: Route Parameter + Query Strings
// URL: GET /api/books/genre/Fantasy?minPrice=10&maxPrice=20&isAvailable=true
[HttpGet("genre/{genre}")]
0 references
public ActionResult<IEnumerable<Book>> GetBooksByGenreWithFilters(
    [FromRoute] string genre,
    [FromQuery] decimal? minPrice,
    [FromQuery] decimal? maxPrice,
    [FromQuery] bool? isAvailable
)
{
    // First filter by genre (mandatory route parameters)
    var filteredBooks = BookData.Books.Where(b => b.Genre.Equals(genre,
        StringComparison.OrdinalIgnoreCase)).AsQueryable();

    // Apply optioanl query string filters
    if (minPrice.HasValue)
    {
        filteredBooks = filteredBooks.Where(b => b.Price >= minPrice.Value);
    }

    if (maxPrice.HasValue)
    {
        filteredBooks = filteredBooks.Where(b => b.Price <= maxPrice.Value);
    }

    if (isAvailable.HasValue)
    {
        filteredBooks = filteredBooks.Where(b => b.IsAvailable == isAvailable.Value);
    }

    var results = filteredBooks.ToList();
```

```
    var results = filteredBooks.ToList();

    if (!results.Any())
        ....
    {
        return NotFound($"No {genre} books match the provided criteria.");
    }

    return Ok(
        new
        {
            Genre = genre,
            Filters = new {minPrice, maxPrice, isAvailable},
            Count = results.Count,
            Books = results
        }
    );
}
```

How it works:

- *[FromRoute]* binds *genre* from URL path (mandatory)
- *[FromQuery]* binds optional filters from query string
- URL structure: */api/books/genre/{genre}?filter1=value1&filter2=value2*

CONVENTION BASED ROUTING (For Comparison)

Let's create *ReportController.cs*

Path: LibraryApi1/Controllers/ReportsController.cs

ReportsController.cs

Controllers > ReportsController.cs > ...

```
using LibraryApi1.Data;
using Microsoft.AspNetCore.Mvc;

namespace LibraryApi1.Controllers
{
    // NO [ApiController] or [Route] attributes here!
    // This controller uses convention-based routing from Program.cs
    0 references
    public class ReportsController : ControllerBase
    {
        // URL: GET /api/reports/sumary
        [HttpGet]
        0 references
        public IActionResult Summary()
        {
            var totalBooks = BookData.Books.Count;
            var availableBooks = BookData.Books.Count(b => b.IsAvailable);

            return Ok(new
            {
                TotalBooks = totalBooks,
                AvailableBooks = availableBooks,
                AvailabilityRate = totalBooks > 0 ?
                    (availableBooks * 100 / totalBooks) : 0
            });
        }

        // URL: GET /api/reports/byauthor/tolkien
        [HttpGet("byauthor/{author}")]
        0 references
        public IActionResult GetBooksByAuthor(string author)
        {
            var books = BookData.Books.Where(b => b.Author.Contains(author,
                StringComparison.OrdinalIgnoreCase)).ToList();

            return Ok(books);
        }
    }
}
```

Update Program.cs for Convention-Based Routing

Add this AFTER app.MapCOntrollers();

Path: LibraryApi1/Program.cs

```
// Add convention-based routing

app.MapControllerRoute(
    name: "default",
    pattern: "api/{controller}/{action}/{id?}"
);
```

Complete Program.cs:

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();   // For attribute routing

// ADD THIS FOR CONVENTION-BASED ROUTING

app.MapControllerRoute(
    name: "default",
    pattern: "api/{controller}/{action}/{id?}"
);

app.Run();
```

IMPORTANT NOTES:

- Controllers with [ApiController] and [Route] use Attribute Routing
- Controllers WITHOUT those attributes use Convention-Based Routing
- Convention routes NOT shown in Swagger

COMPLETE API TESTING RESULTS

Test our API – All Endpoints with Expected Responses

Let's build and run the application. And here we will use the command line or cli. We have to navigate to the exact project folder (LibraryApi1) and run those following commands,

Build the application,

Command: dotnet build

```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi1> dotnet build
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi1 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi1\bin\Debug\net8.0\LibraryApi1.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:03.63
```

To run the application,

Command: dotnet run (we usually get the URL from here to see application response).

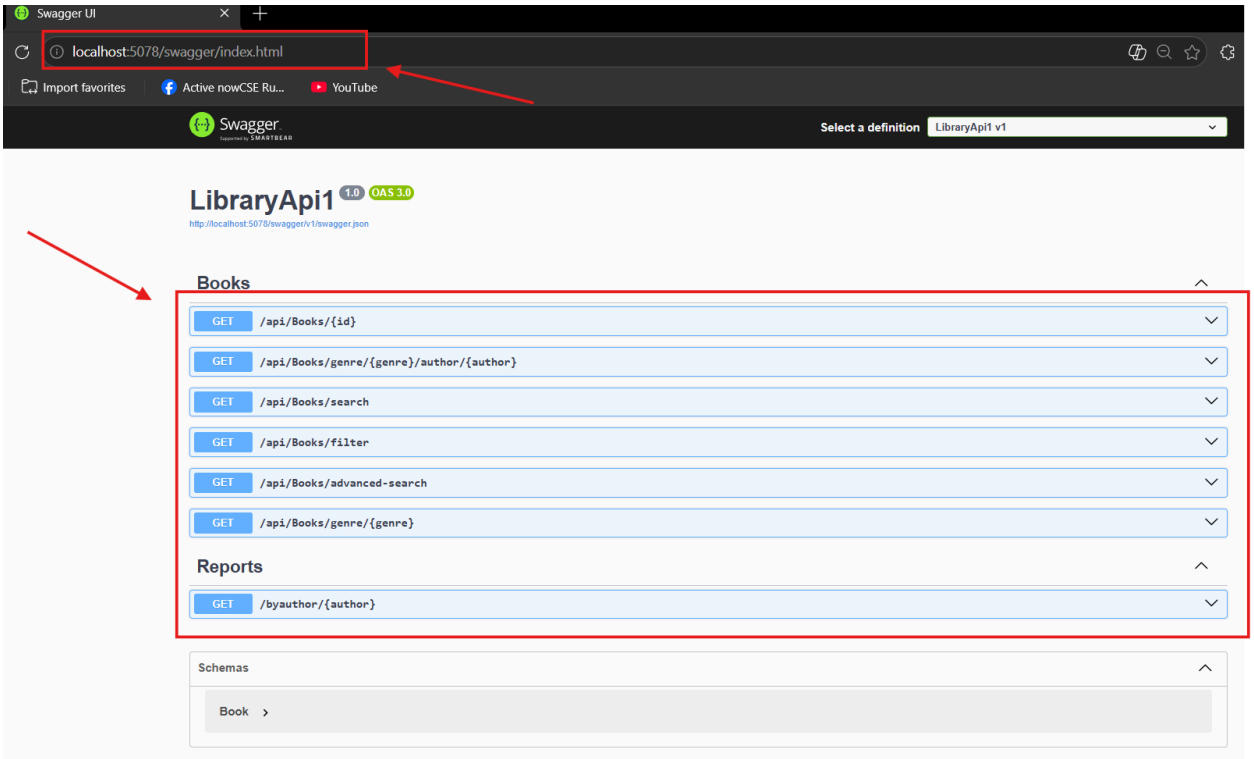
We can run the project and can directly navigate to application response or Swagger UI.

Use can run the command,

Command: dotnet watch run

```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi1> dotnet watch run
dotnet watch 🔥 Hot reload enabled. For a list of supported edits, see https://aka.ms/dotnet/hot-reload
.
  💡 Press "Ctrl + R" to restart.
dotnet watch 🔧 Building...
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi1 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi1\bin\Debug\net8.0\LibraryApi1.dll
dotnet watch 🚀 Started
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5078
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\Routing\LibraryApi1
```

Application Response:



Let's test those Endpoints,

<u>TEST 1: Single Route Parameter {Get Book by ID}</u>

Request: GET **http://localhost:5078/api/Books/3**

Response:

Curl

```
curl -X 'GET' \
  'http://localhost:5078/api/Books/3' \
  -H 'accept: text/plain'
```

Request URL

```
http://localhost:5078/api/Books/3
```

Server response

Code	Details
200	Response body

```
{
  "id": 3,
  "title": "Pride and Prejudice",
  "author": "Jane Austen",
  "genre": "Romance",
  "publishedYear": 1813,
  "isAvailable": true,
  "price": 9.99
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Mon,13 Apr 2026 19:29:26 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links

Request (Not Found): GET **http://localhost:5078/api/Books/99**

Response:

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5078/api/Books/99' \
  -H 'accept: text/plain'
```

Request URL

http://localhost:5078/api/Books/99

Server response

Code	Details
404 *Undocumented*	Error: Not Found

Response body

```
Book with ID 99 not found.
```

Download

Response headers

```
content-type: text/plain; charset=utf-8
date: Mon,13 Apr 2026 19:33:20 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links

<u>TEST 2: Multiple Route Parameters</u>

Request: GET **http://localhost:5078/api/Books/genre/Fantasy/author/J.R.R.%20Tolkien**

Response:

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5078/api/Books/genre/Fantasy/author/J.R.R.%20Tolkien' \
  -H 'accept: text/plain'
```

Request URL

```
http://localhost:5078/api/Books/genre/Fantasy/author/J.R.R.%20Tolkien
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 1,
    "title": "The Hobbit",
    "author": "J.R.R. Tolkien",
    "genre": "Fantasy",
    "publishedYear": 1937,
    "isAvailable": true,
    "price": 15.99
  }
]
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Mon,13 Apr 2026 19:38:19 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links

<u>TEST 3: Single Query String</u>

Request: GET **http://localhost:5078/api/Books/search?genre=Fantasy**

Response:

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5078/api/Books/search?genre=Fantasy' \
  -H 'accept: text/plain'
```

Request URL

```
http://localhost:5078/api/Books/search?genre=Fantasy
```

Server response

Code Details

200

Response body


```
[
  {
    "id": 1,
    "title": "The Hobbit",
    "author": "J.R.R. Tolkien",
    "genre": "Fantasy",
    "publishedYear": 1937,
    "isAvailable": true,
    "price": 15.99
  },
  {
    "id": 5,
    "title": "Harry Potter",
    "author": "J.K. Rowling",
    "genre": "Fantasy",
    "publishedYear": 1997,
    "isAvailable": true,
    "price": 19.99
  }
]
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Mon,13 Apr 2026 19:43:19 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code Description Links

<u>TEST 4: Multiple Query Strings</u>

Request: GET **http://localhost:5078/api/Books/filter?genre=Fantasy&author=J.K.%20Rowling&isAvailable=true**

Response:

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5078/api/Books/filter?genre=Fantasy&author=J.K.%20Rowling&isAvailable=true' \
  -H 'accept: text/plain'
```

Request URL

```
http://localhost:5078/api/Books/filter?genre=Fantasy&author=J.K.%20Rowling&isAvailable=true
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 5,
    "title": "Harry Potter",
    "author": "J.K. Rowling",
    "genre": "Fantasy",
    "publishedYear": 1997,
    "isAvailable": true,
    "price": 19.99
  }
]
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Mon,13 Apr 2026 19:52:04 GMT
server: Kestrel
transfer-encoding: chunked
```

<u>TEST 5: Complex Type Binding with [FromQuery]</u>

Request: GET **http://localhost:5078/api/Books/advanced-search?Genre=Fantasy&Author=J.K.%20Rowling&IsAvailable=true&MinPrice=15&MaxPrice=25**

Response:

Responses

Curl

```
curl -X 'GET' '
  'http://localhost:5078/api/Books/advanced-search?Genre=Fantasy&Author=J.K.%20Rowling&IsAvailable=true&MinPrice=15&MaxPrice=25' \
  -H 'accept: text/plain'
```

Request URL

http://localhost:5078/api/Books/advanced-search?Genre=Fantasy&Author=J.K.%20Rowling&IsAvailable=true&MinPrice=15&MaxPrice=25

Server response

Code	Details
200	Response body


```
[
  {
    "id": 5,
    "title": "Harry Potter",
    "author": "J.K. Rowling",
    "genre": "Fantasy",
    "publishedYear": 1997,
    "isAvailable": true,
    "price": 19.99
  }
]
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Mon,13 Apr 2026 19:57:09 GMT
server: Kestrel
transfer-encoding: chunked
```

<u>TEST 6: Combining Route Parameter + Query Strings</u>

Request: GET **http://localhost:5078/api/Books/genre/Fantasy?minPrice=15&maxPrice=20&isAvailable=true**

Response:

Curl

```
curl -X 'GET' \
  'http://localhost:5078/api/Books/genre/Fantasy?minPrice=15&maxPrice=20&isAvailable=true' \
  -H 'accept: text/plain'
```

Request URL

```
http://localhost:5078/api/Books/genre/Fantasy?minPrice=15&maxPrice=20&isAvailable=true
```

Server response

Code | Details

200

Response body


```
{
  "genre": "Fantasy",
  "filters": {
    "minPrice": 15,
    "maxPrice": 20,
    "isAvailable": true
  },
  "count": 2,
  "books": [
    {
      "id": 1,
      "title": "The Hobbit",
      "author": "J.R.R. Tolkien",
      "genre": "Fantasy",
      "publishedYear": 1937,
      "isAvailable": true,
      "price": 15.99
    },
    {
      "id": 5,
      "title": "Harry Potter",
      "author": "J.K. Rowling",
      "genre": "Fantasy",
      "publishedYear": 1997,
      "isAvailable": true,
      "price": 19.99
    }
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Mon,13 Apr 2026 20:02:25 GMT
server: Kestrel
transfer-encoding: chunked
```

<u>TEST 7: Convention-Based Routing (ReportsController)</u>

Request: GET **http://localhost:5078/api/Reports/Summary**

Response (200 OK):

localhost:5078/api/Reports/Summary

Import favorites [redacted] YouTube

Pretty-print


```
{"totalBooks":5,"availableBooks":4,"availabilityRate":80}
```

Request: GET **http://localhost:5078/byauthor/J.R.R.%20Tolkien**

Response (200 OK):

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5078/byauthor/J.R.R.%20Tolkien' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:5078/byauthor/J.R.R.%20Tolkien
```

Server response

Code | Details

200

Response body

```
[
  {
    "id": 1,
    "title": "The Hobbit",
    "author": "J.R.R. Tolkien",
    "genre": "Fantasy",
    "publishedYear": 1937,
    "isAvailable": true,
    "price": 15.99
  }
]
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Mon,13 Apr 2026 20:14:56 GMT
server: Kestrel
transfer-encoding: chunked
```

To close the application, we have to press “Ctrl+c” in the terminal, or there may be GUI options due to IDEs.

```
        Application is shutting down...
  dotnet watch 🔴 Shutdown requested. Press Ctrl+C again to force exit.

PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi1>
```